

Building a reading platform based on Bookmate

Salazar M. Andrés F., Delgado M. Ichel A.
Universidad Distrital Francisco José de Caldas
Department of Systems Engineering

ABSTRACT

This project presents a hybrid database design for a digital reading platform supporting books, audiobooks, user interaction, and recommendations. PostgreSQL and MongoDB were combined to manage structured and unstructured data. Key features were tested with realistic scenarios, showing strong performance and scalability.

GOAL

This project aims to design and implement a normalized relational database model for a social reading platform. The expected outcome is a scalable schema that supports structured and unstructured content, social interaction, and personalized recommendations.

PROPOSED SOLUTION

The proposed solution is a hybrid architecture combining PostgreSQL for structured data (e.g., users, books, reviews) and MongoDB for binary content (e.g., PDFs and MP3s). The schema includes entities for reading progress, recommendations via similarity, and social features like following users. Logical deletion, UUIDs, and query optimization through indexing were used to support integrity and performance. The design for the E-R diagram includes core entities for users, books, reading progress, reviews, social connections, and recommendations. Relationships are managed via normalized tables, and unstructured content is stored externally using MongoDB.

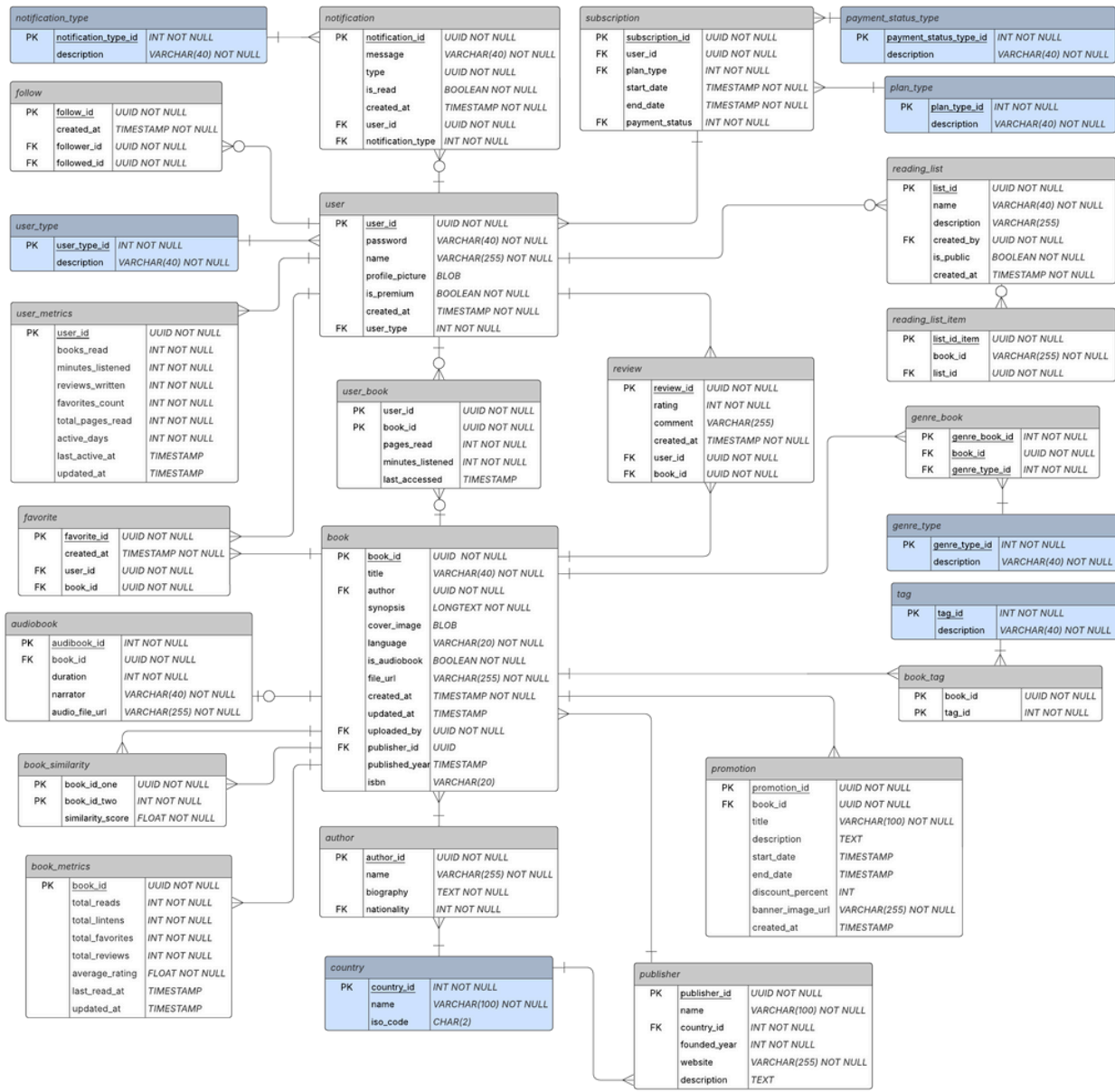


Figure 1. Proposed E-R diagram.

EXPERIMENTS

- A dataset of several hundred records was generated using Python and the Faker library. Data was inserted across all entities, ensuring that referential integrity and business constraints were respected. Manual queries were executed to validate each user story.
- Example test cases included: retrieving personalized book recommendations, excluding books already read; fetching books read by followed users; accessing book metadata and associated content in MongoDB. Each query was tested by verifying the accuracy of its output and measuring execution time.

Query Goal	Description
Book listing (paginated)	20 books with metadata
Recommendations by similarity	10 books not read, ordered by score
Social-based recommendations	Books read by followed users (13k rows)

Table 1. Some of the test cases.

RESULTS

The proposed relational model was validated through queries derived from core user stories, such as retrieving book listings, generating recommendations, tracking reading progress, and managing favorites or followers. All queries were manually verified using realistic datasets generated with the Faker library. One of the most comprehensive queries demonstrates the ability to search for both books and audiobooks in a single result. It supports case-insensitive filtering by title, author name, or genre, aggregates genre labels per book, includes pagination through `LIMIT` and `OFFSET`, and provides the total number of results using a window function (`COUNT(\*) OVER()`).

```
SELECT
  b.book_id,
  b.title,
  a.name AS author_name,
  b.synopsis,
  b.language,
  b.cover_image,
  CASE
    WHEN b.is_audiobook THEN NULL
    ELSE 'https://url-to-book-content/' || b.
book_id || '?expires=' || EXTRACT(EPOCH FROM NOW
() + INTERVAL '1 hour')
END AS online_reader_url,
b.published_year,
p.name AS publisher_name
FROM
  book b
JOIN
  author a ON b.author = a.author_id
LEFT JOIN
  publisher p ON b.publisher_id = p.publisher_id
WHERE
  b.book_id = '9e8de2f8-ff78-9337-feeafbe5fa933b19'
  AND b.is_audiobook = FALSE
  AND b.is_active = TRUE;
```

Figure 2. Query to filter books with pagination.

This confirms the schema's expressiveness and its ability to handle complex queries efficiently. In addition, queries for personalized recommendations were implemented using both similarity scores and the reading activity of followed users. These mechanisms allow the platform to tailor suggestions based on user behavior and social interaction, fulfilling the expected functional scope.

Query Goal	Description	Result Time
Book listing (paginated)	20 books with metadata	1.726s
Recommendations by similarity	10 books not read, ordered by score	0.006s
Social-based recommendations	Books read by followed users (13k rows)	0.264s

Table 2. Results of query tests.

The system demonstrated strong performance with both specific and large-scale queries. Logical deletion using the `is\_active` flag worked effectively, and MongoDB integration for binary content retrieval was successful. Overall, the schema is flexible, efficient, and supports complex queries while preserving normalization and data integrity.

CONCLUSIONS

The database architecture met all project objectives, enabling core features like content management, recommendations, and user interaction. Tests based on real user stories confirmed high performance and correct behavior. The combined use of PostgreSQL and MongoDB proved effective for managing structured and unstructured data. The platform is scalable and ready for future enhancements such as gamification, analytics, and AI-based recommendations.

REFERENCES

- [1] E. SolutionsHub, "Exploring Business Model Canvas examples". Oct. 20, 2023. Available: [https://solutionshub.epam.com/blog/post/business-model-canvas-examples\\[0.lin](https://solutionshub.epam.com/blog/post/business-model-canvas-examples\[0.lin)
- [2] IBM, "Relational Databases," IBM.com, Oct. 20, 2021. [https://www.ibm.com/think/topics/relational-databases\\[0.lin](https://www.ibm.com/think/topics/relational-databases\[0.lin)
- [3] Lucidchart, "Database Design Structure - Schema Tutorial", 2025. [https://www.lucidchart.com/pages/tutorial/database-design-and-structure\\[0.lin](https://www.lucidchart.com/pages/tutorial/database-design-and-structure\[0.lin)

Building a reading platform based on Bookmate

Salazar M. Andrés F., Delgado M. Ichel A.
Universidad Distrital Francisco José de Caldas
Departament of Systems Engineering

ABSTRACT

This project presents the design of the database layer for a digital reading platform inspired by Bookmate. We developed a Business Model Canvas to identify core features, then defined functional requirements based on user stories. A normalized relational schema was created to support subscriptions, user interactions, and content management. The database architecture defines how data is collected and served to other system layers, laying the foundation for future integration and system development.

GOAL

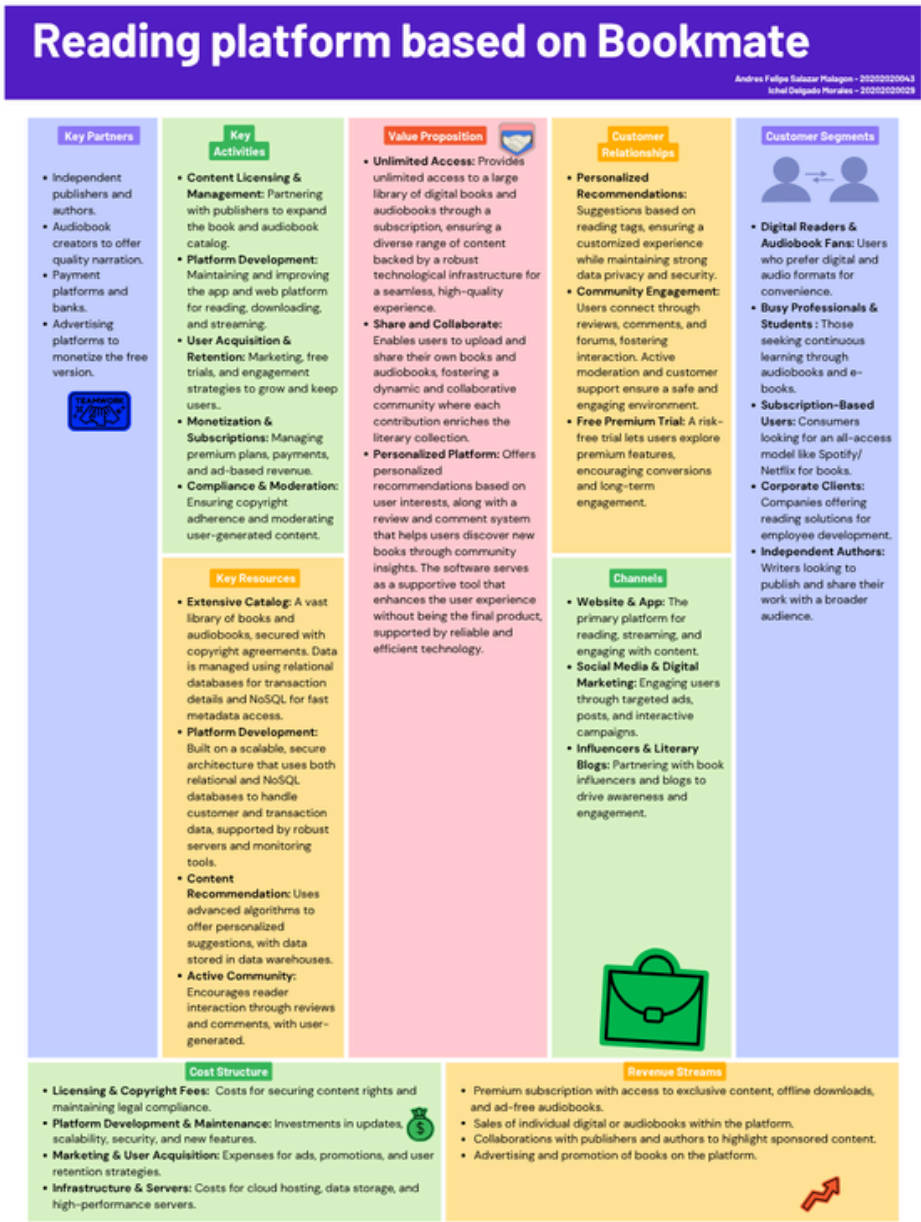
This project aims to design and implement a normalized relational database model for a social reading platform. The expected outcome is a scalable schema that supports structured and unstructured content, social interaction, and personalized recommendations.

METHODS

We started by defining the Business Model Canvas to understand the platform's structure and data needs. From user stories, we derived functional requirements and used them to design a normalized relational schema supporting key features like subscriptions, reviews, and reading lists. A preliminary architecture was also created to define how data enters the system and is made available to external layers.

RESULTS

- Business Model Canvas:** Emphasizes platform monetization via subscription plans and community engagement.



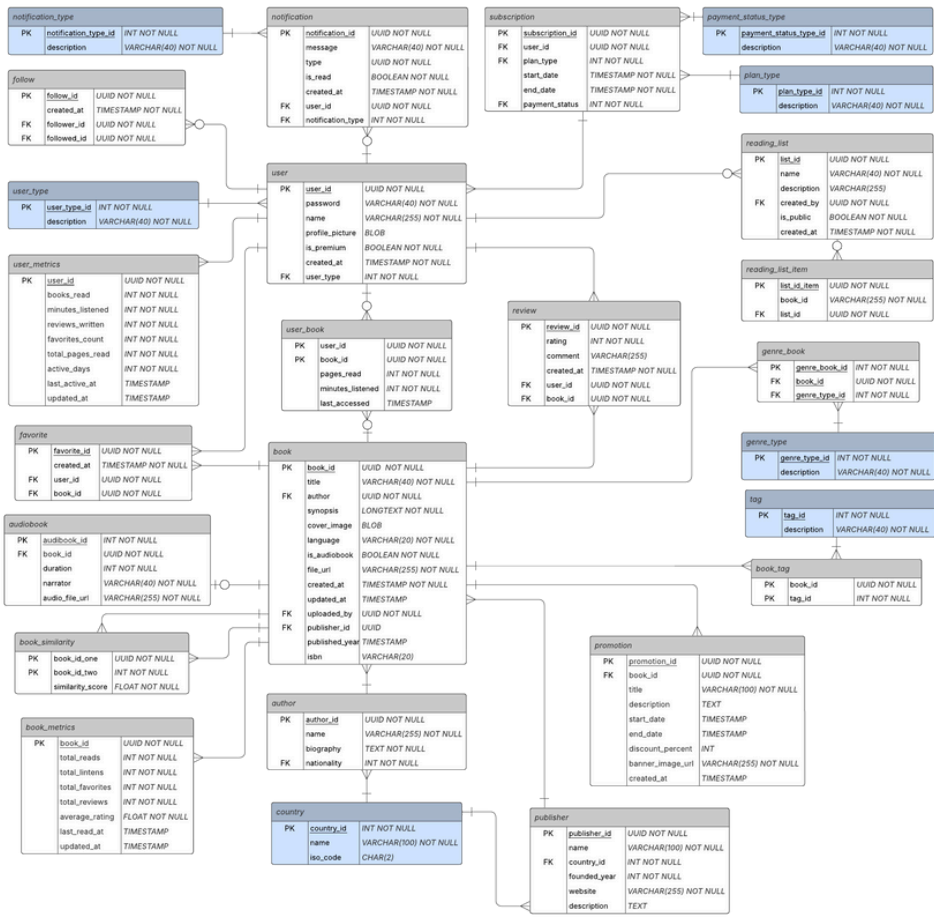
- Requirements Gathering:** Functional and non-functional requirements were derived from detailed user stories covering readers, content creators, and administrators.

Code	Requirement
FR-	
FR-	
FR-	

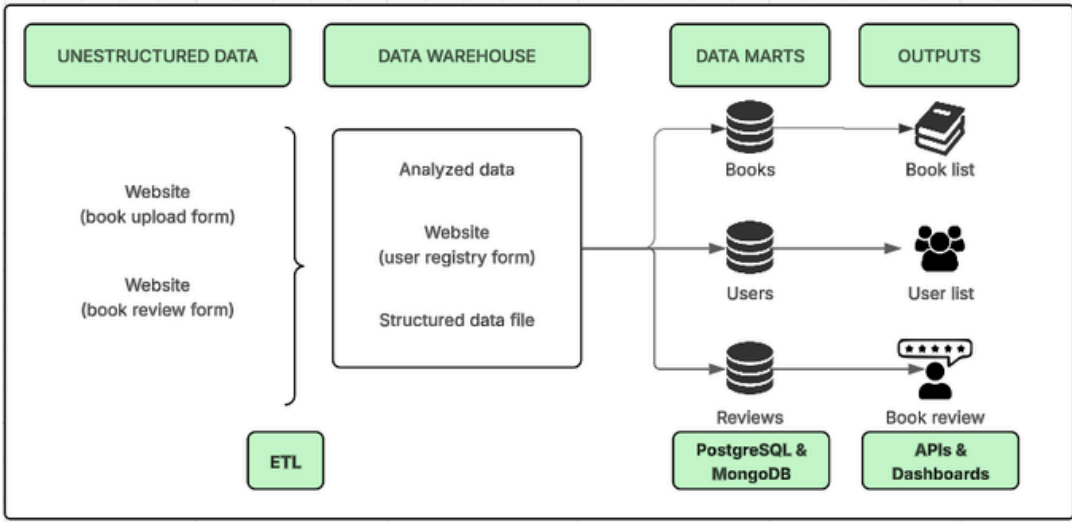
- Validated Use Cases:** Subscription flow, user interactions (follows, reviews), reading list creation, and notifications.

Title:	Priority:
User Story:	
Acceptance Criteria:	

- Relational Schema:**
 - Core entities: user, book, subscription, review, favorite, reading_list, follow, notification.
 - Intermediate tables and foreign key constraints for many-to-many relationships.



- Preliminary Architecture Design:** A high-level system architecture was proposed, incorporating distributed storage, content delivery, and scalable user management to handle big data volumes and high concurrency.



CONCLUSIONS

The project defined the database layer for a reading platform inspired by Bookmate, based on user stories and functional requirements. The relational model supports subscriptions, user content, and personalized lists. The architecture ensures clean data flow and scalability. Future work will focus on populating the database, optimizing queries, and integrating with the backend.

REFERENCES

[1] E. SolutionsHub, "Exploring Business Model Canvas examples". Oct. 20, 2023. Available: [https://solutionshub.epam.com/blog/post/business-model-canvas-examples\\\[0.ln\]](https://solutionshub.epam.com/blog/post/business-model-canvas-examples\\[0.ln])

[2] IBM, "Relational Databases," Ibm.com, Oct. 20, 2021. [https://www.ibm.com/think/topics/relational-databases\\\[0.ln\]](https://www.ibm.com/think/topics/relational-databases\\[0.ln])

[3] Lucidchart, "Database Design Structure - Schema Tutorial", 2025. [https://www.lucidchart.com/pages/tutorial/database-design-and-structure\\\[0.ln\]](https://www.lucidchart.com/pages/tutorial/database-design-and-structure\\[0.ln])